Machine Coding: Accessible Modal (Open / Close)

- **Problem:** Build a reusable `Modal` component and use it twice: an "Edit profile" form and a "Delete account?" confirmation. It opens from a button and closes with the × button, Cancel, Escape, or a click on the backdrop (configurable). While open, focus moves inside and is trapped there, the page behind can't be scrolled, clicked or reached by screen readers, and on close focus returns to the button that opened it.
- **Tech stack:** React 19 + Vite (JavaScript, plain CSS)
- **Time limit:** 60 minutes

PART A: Thought process

General steps for any machine coding task

1. **Clarify** the requirements. Don't guess; ask.
2. **List the interactions.** Every user action and what it should do becomes your checklist.
3. **Sketch the architecture.** Components, hooks and modules, with one job each.
4. **Design the state.** Find the minimum you have to store; derive the rest.
5. **Define the data and contracts.** Example data and the API shape (mock it if there's no API).
6. **Build the simplest working version first,** then add one working layer at a time.
7. **Handle every state and edge case:** empty, loading, error, success.
8. **Wrap up.** Explain the trade-offs and what you'd add with more time.

Time budget (60 min)

Phase	Time	Output
Requirements + clarifying questions	5 min	Close rules, a11y needs decided
Use cases + API (props) + state	5 min	`Modal` contract, one `openModal` state
Open/close + backdrop + × (simplest version)	8 min	Working modal
Portal, Escape, backdrop click done right	10 min	Correct closing
Focus in / trap / restore, inert background, scroll lock	15 min	Accessible modal
Reuse it (form + confirm), styles, animation	7 min	Polished
Keyboard-only test + wrap-up	10 min	Demo + trade-offs

Tip: think out loud

The interviewer is grading how you reason, not just the final code. Say each decision as you make it: *"A modal is mostly about focus. When it opens, focus must go inside; Tab must not escape to the page behind; Escape closes it; and when it closes, focus goes back to the button that opened it. Otherwise keyboard and screen-reader users get lost."* Then demo it with the keyboard only.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Button opens the modal; × / Cancel / Escape close it; a backdrop click closes it unless disabled (`closeOnBackdrop={false}` for destructive confirms).
- A reusable `Modal` with a title, optional description and any content (children).
- Two uses: the Edit profile form (validates, saves, closes) and the Delete confirmation.
- The page behind doesn't scroll while the modal is open.
- Focus: moves into the modal on open (to a chosen field, or the first focusable), stays trapped inside, returns to the trigger on close.

Non-functional requirements

Concern	What it means here
Performance	Nothing is rendered when closed (`return null`); listeners exist only while open.
Scalability	One `Modal` used anywhere; one `openModal` value means two modals can't be open by accident.
Error handling	Form validation inside the modal; closing only via `onClose` (the parent stays in control).
Accessibility	`role="dialog"`, `aria-modal="true"`, `aria-labelledby` / `aria-describedby`, focus trap, Escape, focus restore, `inert` background, visible focus, reduced-motion support.
Reusability	`Modal` knows nothing about forms or deletion; content is passed as `children`. Behaviour lives in `useModalEffects`.
Extensibility	Sizes, footers, nested modals, close animations or a native `<dialog>` version can be added behind the same props.

Clarifying questions to ask the interviewer

1. Which ways should close it: ×, Escape, backdrop? Should a destructive confirm ignore backdrop clicks?
2. Where should focus go when it opens: the first field, the safest button (Cancel), or the dialog itself?
3. Can modals be nested (a confirm on top of a form)?
4. Should it be controlled by the parent (`isOpen`) or manage its own state?
5. Do we need open/close animations? Respect "reduce motion"?
6. Can I use the native `<dialog>` element, or should I build it from scratch?

Assumptions we will build with

- Controlled component: `isOpen` + `onClose` from the parent.
- Closes with ×, Escape and backdrop (backdrop off for the delete confirm).
- Focus goes to `[data-autofocus]` if present (the name field / the Cancel button), otherwise the first focusable element.
- Rendered in a portal on `document.body`; the app root gets `inert` while open.

- One modal open at a time; open animation only (with `prefers-reduced-motion`).
- Built from scratch (the interview point), with `<dialog>` discussed as the production alternative.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Click "Edit profile"	Modal opens; focus in the name field; page behind can't scroll
Press Tab repeatedly	Focus cycles × → name → Cancel → Save → × … never reaches the page
Press Shift+Tab on the first element	Focus wraps to the last element
Press Escape	Modal closes; focus returns to "Edit profile"
Click the backdrop (Edit)	Closes
Press the mouse inside the modal, drag, release on the backdrop	Does **not** close (e.g. while selecting text)
Click the backdrop (Delete)	Stays open (`closeOnBackdrop={false}`), focus stays inside
Save with an empty name	"Name is required", modal stays open
Save a valid name	Name updated, "Profile saved." announced, modal closes
Reopen after Cancel	The form shows the saved name (the draft is not kept)
Click "Delete account"	Modal opens with focus on **Cancel** (the safe default)
Screen reader	Announces "Edit profile, dialog"; the page behind isn't read
Long content in the modal	The modal scrolls inside; max height = viewport
Phone width	Modal shrinks to the screen width with padding

3. Design the architecture

Module	Responsibility
`App`	Owns `openModal` ("edit" / "delete" / null) and the profile name; renders the triggers and the two modals
`components/Modal.jsx`	Portal + backdrop + dialog markup (role, labels, × button); backdrop-click rule
`hooks/useModalEffects.js`	Side effects while open: initial focus, Tab trap, Escape, scroll lock, `inert`, focus restore
`components/EditProfileForm.jsx`	The form inside the modal: draft name, validation, Save / Cancel

```
App  (openModal, name)
├── trigger buttons
├── <Modal isOpen={openModal==="edit"} onClose={close} title="Edit profile">
│     └── <EditProfileForm initialName onSave onCancel />
```

```
└── <Modal isOpen={openModal==="delete"} onClose={close} closeOnBackdrop={false} …>
      └── Cancel / Delete buttons
Modal ──createPortal──▶ document.body
  └── useModalEffects(dialogRef, isOpen, onClose)
```

Start simple. First version: `{isOpen && <div className="backdrop"><div className="modal">…</div></div>}` with a close button. Then add the portal, Escape, focus handling, and finally extract the effects into a hook.

4. Design the state / data model

State	Where	Purpose
`openModal`	App	`null | "edit" | "delete"`: which modal is open
`name`	App	Saved profile name
`message`	App	Live status message ("Profile saved.")
`name` (draft), `error`	EditProfileForm	The draft being edited and its validation error
`onCloseRef`, `pressStartedOnBackdrop` (refs)	Modal / hook	Latest callback; where the mouse press started

```
// App
const [openModal, setOpenModal] = useState(null); // one value, not one boolean per modal
const close = () => setOpenModal(null);

<Modal isOpen={openModal === "edit"} onClose={close} title="Edit profile">
  <EditProfileForm initialName={name} onCancel={close} onSave={(n) => { setName(n); close(); }} />
</Modal>
```

Senior note

- **Controlled modal:** the parent owns `isOpen`; the modal only *asks* to close (`onClose`). The parent can then veto it (unsaved changes) or run logic on close.
- **One `openModal` value instead of `isEditOpen` + `isDeleteOpen`:** two modals can't be open at the same time by mistake.
- **The draft lives in the form, not in `App`:** the form mounts when the modal opens, so the draft resets automatically each time, and Cancel needs no cleanup.
- **Refs for non-visual bookkeeping:** the previously focused element, the latest `onClose`, and where a mouse press started don't need re-renders.

5. Design the data structures and contracts

Modal props (the component API):

```
<Modal
  isOpen={boolean}            // required, controlled by the parent
```

```
  onClose={() => void}        // called on ×, Escape, backdrop
  title="Edit profile"        // used for aria-labelledby
  description="…"             // optional, aria-describedby
  closeOnBackdrop={true}      // false for destructive confirms
>
  {children}                  // anything; mark the field to focus with data-autofocus
</Modal>
```

Hook contract:

```
useModalEffects(dialogRef, isOpen, onClose)
// while open: focus [data-autofocus] | first focusable | dialog; trap Tab; Escape → onClose;
// lock body scroll; set inert on #root; on close: undo all + restore focus
```

Focusable elements selector (what Tab can reach):

```
'a[href], button:not([disabled]), input:not([disabled]), select:not([disabled]),
 textarea:not([disabled]), [tabindex]:not([tabindex="-1"])'
```

6. Trace the data flow

User opens Edit profile, then presses Escape:

```
Click "Edit profile"  (document.activeElement = that button)
  ↓ setOpenModal("edit") → App re-renders → <Modal isOpen> renders into document.body (portal)
  ↓ useModalEffects (isOpen = true):
  ↓   remember previouslyFocused = "Edit profile" button
  ↓   body overflow hidden (+ scrollbar padding), #root gets inert
  ↓   focus [data-autofocus] → the name input
  ↓   add keydown listener (Tab trap + Escape)
Press Escape
  ↓ keydown → onCloseRef.current() → close() → setOpenModal(null)
  ↓ Modal returns null → effect cleanup runs:
  ↓   remove listener, restore body styles, remove inert, previouslyFocused.focus()
  ↓ UI: modal gone, focus ring back on "Edit profile"
```

7. Important technical challenges

Challenge 1: closing correctly (Escape and backdrop)

- *Problem:* The first version closes on `e.target.className == "model_container"`. That breaks as soon as the class name changes or another class is added, has no Escape key, and a user who selects text inside the modal and releases the mouse over the backdrop closes it by accident.
- *Solution:* Compare `e.target === e.currentTarget` (the click was on the backdrop itself), require the press to have *started* on the backdrop, and listen for Escape while open.

```
const onBackdropMouseDown = (e) => {
  pressStartedOnBackdrop.current = e.target === e.currentTarget;
  if (pressStartedOnBackdrop.current) e.preventDefault(); // keep focus inside the modal
};
const onBackdropClick = (e) => {
  if (closeOnBackdrop && pressStartedOnBackdrop.current && e.target === e.currentTarget) onClose();
};
```

Challenge 2: focus management (in, trapped, back)

- *Problem:* After opening, focus stays on the button behind the modal: Tab walks through the hidden page, screen readers read the page instead of the dialog, and after closing, focus is lost (it jumps to the top of the page).
- *Solution:* Save the active element, focus inside, wrap Tab / Shift+Tab at the edges, and restore focus in the cleanup.

```
const previouslyFocused = document.activeElement;
(dialog.querySelector("[data-autofocus]") ?? dialog.querySelector(FOCUSABLE) ?? dialog).focus();
// keydown Tab:
if (e.shiftKey && document.activeElement === first) { e.preventDefault(); last.focus(); }
else if (!e.shiftKey && document.activeElement === last) { e.preventDefault(); first.focus(); }
// cleanup:
previouslyFocused?.focus?.();
```

Challenge 3: stacking, scrolling and the page behind

- *Problem:* `position: absolute` inside the app can end up *under* other content (a parent with `overflow: hidden`, `transform` or a lower `z-index`), and the page still scrolls behind the modal. Locking scroll removes the scrollbar, so the page jumps sideways.
- *Solution:* Render into `document.body` with a portal and `position: fixed; inset: 0`; lock `body` overflow and pad by the scrollbar width; make `#root` `inert` (not clickable, not focusable, hidden from assistive tech).

```
createPortal(<div className="backdrop">…</div>, document.body);
const scrollbar = window.innerWidth - document.documentElement.clientWidth;
document.body.style.overflow = "hidden";
if (scrollbar > 0) document.body.style.paddingRight = `${scrollbar}px`;
appRoot?.setAttribute("inert", "");
```

Challenge 4: stale `onClose` vs re-running the effect

- *Problem:* The parent passes `onClose={close}`, a new function on every render. If `onClose` is in the effect's dependency list, the effect re-runs on **every keystroke** in the form: it re-focuses the first field, re-locks scroll, and the trap resets. If it's left out, the Escape handler calls a stale callback.
- *Solution:* Keep the latest callback in a ref, and run the big effect only when `isOpen` changes.

```
const onCloseRef = useRef(onClose);
useEffect(() => { onCloseRef.current = onClose; });   // every render, cheap
useEffect(() => { /* … onCloseRef.current() on Escape … */ }, [isOpen, dialogRef]);
```

8. Implement in the correct order

#	Step	Working result
1	`isOpen` state + conditional backdrop/modal markup + × button	Opens and closes
2	Backdrop click with `target === currentTarget`	Click outside closes
3	`createPortal` to `document.body`, `position: fixed`	Always on top
4	Extract `Modal` with props (`isOpen`, `onClose`, `title`, `children`)	Reusable
5	`role="dialog"`, `aria-modal`, `aria-labelledby` via `useId`	Announced correctly
6	Escape listener while open	Keyboard close
7	Initial focus (`data-autofocus` / first focusable) + restore on close	Focus in and back
8	Tab / Shift+Tab trap	Focus can't escape
9	Scroll lock with scrollbar compensation + `inert` on `#root`	Page behind is frozen
10	`onClose` ref pattern; press-started-on-backdrop rule	No refocus on typing; no accidental close
11	Second use: confirm dialog with `closeOnBackdrop={false}` + focus on Cancel	Reuse proven
12	Styles, max-height scroll, animation + `prefers-reduced-motion`	Polished

9. Cover all states and edge cases

State / edge case	Handling
Closed	Nothing rendered; no listeners
Opening	Portal mounted; focus moved in; scroll locked; root inert
Open, typing in the form	Effect doesn't re-run (ref pattern); focus stays where the user is
Tab at the last / Shift+Tab at the first element	Wraps around
No focusable element inside	The dialog itself (`tabIndex={-1}`) gets focus; Tab is blocked
Escape	Closes (via `onClose`)
Backdrop click	Closes unless `closeOnBackdrop={false}`
Drag from inside to the backdrop	Doesn't close
Destructive confirm	Focus on Cancel; backdrop disabled
Invalid form submit	Error shown; stays open

State / edge case	Handling
Closing	Styles restored, inert removed, focus back on the trigger
Trigger removed while open	`previouslyFocused?.focus?.()` doesn't throw
Content taller than the screen	Modal scrolls internally (`max-height` + `overflow-y: auto`)
Page had a scrollbar	Padding compensates; no layout jump
Reduced motion	Animations off
React StrictMode (effects run twice in dev)	The cleanup fully undoes the effect, so the second run is correct

10. Final structure

```
_09_model_open_features/
├── index.html                    ← <div id="root"> (made inert while a modal is open)
└── src/
    ├── main.jsx
    ├── index.css                 ← global reset
    ├── App.jsx                   ← openModal state, triggers, two modal uses
    ├── App.css                   ← page, backdrop, modal, animation
    ├── hooks/
    │   └── useModalEffects.js    ← focus in/trap/restore, Escape, scroll lock, inert
    └── components/
        ├── Modal.jsx             ← portal + dialog markup + backdrop rule
        └── EditProfileForm.jsx   ← form content for the edit modal
```

11. Full working code

`src/hooks/useModalEffects.js`

```
import { useEffect, useRef } from "react";

const FOCUSABLE =
  'a[href], button:not([disabled]), input:not([disabled]), select:not([disabled]),
textarea:not([disabled]), [tabindex]:not([tabindex="-1"])';

// Everything a modal must do while open: focus in, trap Tab, Escape to close,
// lock page scroll, make the page behind inert, and give focus back on close.
export function useModalEffects(dialogRef, isOpen, onClose) {
  const onCloseRef = useRef(onClose);
  useEffect(() => {
    onCloseRef.current = onClose; // latest callback without re-running the big effect below
  });

  useEffect(() => {
    if (!isOpen) return;
```

```
    const dialog = dialogRef.current;
    const previouslyFocused = document.activeElement;
    const appRoot = document.getElementById("root");

    // Lock scroll; pad by the scrollbar width so the page doesn't jump sideways.
    const { overflow, paddingRight } = document.body.style;
    const scrollbar = window.innerWidth - document.documentElement.clientWidth;
    document.body.style.overflow = "hidden";
    if (scrollbar > 0) document.body.style.paddingRight = `${scrollbar}px`;
    appRoot?.setAttribute("inert", ""); // page behind: not clickable, not focusable, hidden from screen
readers

    (dialog.querySelector("[data-autofocus]") ?? dialog.querySelector(FOCUSABLE) ?? dialog).focus();

    const onKeyDown = (e) => {
      if (e.key === "Escape") {
        e.stopPropagation();
        onCloseRef.current();
        return;
      }
      if (e.key !== "Tab") return;
      const items = [...dialog.querySelectorAll(FOCUSABLE)];
      if (items.length === 0) {
        e.preventDefault();
        return;
      }
      const first = items[0];
      const last = items[items.length - 1];
      if (e.shiftKey && (document.activeElement === first || !dialog.contains(document.activeElement)))
{
        e.preventDefault();
        last.focus();
      } else if (!e.shiftKey && document.activeElement === last) {
        e.preventDefault();
        first.focus();
      }
    };
    document.addEventListener("keydown", onKeyDown);

    return () => {
      document.removeEventListener("keydown", onKeyDown);
      document.body.style.overflow = overflow;
      document.body.style.paddingRight = paddingRight;
      appRoot?.removeAttribute("inert");
      previouslyFocused?.focus?.(); // back to the button that opened the modal
    };
```

```
  }, [isOpen, dialogRef]);
}
```

src/components/Modal.jsx

```
import { useId, useRef } from "react";
import { createPortal } from "react-dom";
import { useModalEffects } from "../hooks/useModalEffects";

// Reusable, controlled modal. The parent owns `isOpen`; the modal only asks to close via onClose().
const Modal = ({ isOpen, onClose, title, description, children, closeOnBackdrop = true }) => {
  const dialogRef = useRef(null);
  const pressStartedOnBackdrop = useRef(false);
  const titleId = useId();
  const descriptionId = useId();

  useModalEffects(dialogRef, isOpen, onClose);

  if (!isOpen) return null;

  // Close only when the click both started and ended on the backdrop,
  // so selecting text inside the modal and releasing outside doesn't close it.
  const onBackdropMouseDown = (e) => {
    pressStartedOnBackdrop.current = e.target === e.currentTarget;
    if (pressStartedOnBackdrop.current) e.preventDefault(); // keep focus inside the modal
  };
  const onBackdropClick = (e) => {
    if (closeOnBackdrop && pressStartedOnBackdrop.current && e.target === e.currentTarget) onClose();
  };

  return createPortal(
    <div className="backdrop" onMouseDown={onBackdropMouseDown} onClick={onBackdropClick}>
      <div
        ref={dialogRef}
        className="modal"
        role="dialog"
        aria-modal="true"
        aria-labelledby={titleId}
        aria-describedby={description ? descriptionId : undefined}
        tabIndex={-1}
      >
        <header className="modal-header">
          <h2 id={titleId}>{title}</h2>
          <button type="button" className="icon-btn" onClick={onClose} aria-label="Close">
            ×
          </button>
        </header>
```

```
        {description && (
          <p id={descriptionId} className="modal-description">
            {description}
          </p>
        )}
        <div className="modal-body">{children}</div>
      </div>
    </div>,
    document.body
  );
};

export default Modal;
```

src/components/EditProfileForm.jsx

```
import { useState } from "react";

// Mounted only while the modal is open, so the draft resets every time it opens.
const EditProfileForm = ({ initialName, onSave, onCancel }) => {
  const [name, setName] = useState(initialName);
  const [error, setError] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!name.trim()) {
      setError("Name is required");
      return;
    }
    onSave(name.trim());
  };

  return (
    <form onSubmit={handleSubmit} noValidate>
      <label htmlFor="profile-name">Display name</label>
      <input
        id="profile-name"
        data-autofocus
        value={name}
        onChange={(e) => {
          setName(e.target.value);
          setError("");
        }}
        aria-invalid={Boolean(error)}
        aria-describedby={error ? "profile-name-error" : undefined}
      />
      {error && (
```

```
        <p id="profile-name-error" role="alert" className="error">
          {error}
        </p>
      )}
      <div className="modal-actions">
        <button type="button" className="secondary" onClick={onCancel}>
          Cancel
        </button>
        <button type="submit">Save</button>
      </div>
    </form>
  );
};

export default EditProfileForm;
```

src/App.jsx

```
import { useState } from "react";
import "./App.css";
import Modal from "./components/Modal";
import EditProfileForm from "./components/EditProfileForm";

function App() {
  const [openModal, setOpenModal] = useState(null); // null | "edit" | "delete": one modal at a time
  const [name, setName] = useState("Ashutosh");
  const [message, setMessage] = useState("");
  const close = () => setOpenModal(null);

  return (
    <main className="container">
      <h1>Modal Open and Close</h1>
      <p className="lead">
        Hello, <strong>{name}</strong>
      </p>

      <div className="page-actions">
        <button type="button" onClick={() => setOpenModal("edit")}>
          Edit profile
        </button>
        <button type="button" className="danger" onClick={() => setOpenModal("delete")}>
          Delete account
        </button>
      </div>
      <p aria-live="polite" className="message">
        {message}
      </p>
```

```
      {/* Long content so you can check that the page behind doesn't scroll */}
      {Array.from({ length: 30 }, (_, i) => (
        <p key={i} className="filler">
          Page content line {i + 1}
        </p>
      ))}

      <Modal isOpen={openModal === "edit"} onClose={close} title="Edit profile">
        <EditProfileForm
          initialName={name}
          onCancel={close}
          onSave={(newName) => {
            setName(newName);
            setMessage("Profile saved.");
            close();
          }}
        />
      </Modal>

      <Modal
        isOpen={openModal === "delete"}
        onClose={close}
        title="Delete account?"
        description="This cannot be undone."
        closeOnBackdrop={false}
      >
        <div className="modal-actions">
          <button type="button" className="secondary" onClick={close} data-autofocus>
            Cancel
          </button>
          <button
            type="button"
            className="danger"
            onClick={() => {
              setMessage("Account deleted (demo).");
              close();
            }}
          >
            Delete
          </button>
        </div>
      </Modal>
    </main>
  );
}
```

```
export default App;
```

src/App.css

```
body { background: #f3f4f6; font-family: system-ui, sans-serif; color: #111827; }
.container { max-width: 640px; margin: 32px auto; padding: 20px; background: white; border-radius: 10px;
}
h1 { text-align: center; font-size: 24px; margin-bottom: 12px; }
.lead { text-align: center; margin-bottom: 16px; }
.page-actions { display: flex; gap: 12px; justify-content: center; }
.message { text-align: center; min-height: 24px; margin: 12px 0; color: #15803d; }
.filler { color: #9ca3af; padding: 6px 0; border-bottom: 1px dashed #e5e7eb; }

button { padding: 10px 18px; border: none; border-radius: 8px; background: #4f46e5; color: white; font-
size: 15px; cursor: pointer; }
button:hover { filter: brightness(0.95); }
button:focus-visible, input:focus-visible { outline: 3px solid #a5b4fc; outline-offset: 2px; }
button.secondary { background: #e5e7eb; color: #111827; }
button.danger { background: #dc2626; }

.backdrop { position: fixed; inset: 0; background: rgba(17, 24, 39, 0.55); display: grid; place-items:
center; padding: 16px; z-index: 1000; animation: fade-in 150ms ease-out; }
.modal { width: min(440px, 100%); max-height: calc(100vh - 32px); overflow-y: auto; background: white;
border-radius: 12px; padding: 20px; box-shadow: 0 20px 40px rgba(0, 0, 0, 0.25); animation: pop-in 150ms
ease-out; }
.modal:focus { outline: none; }
.modal-header { display: flex; justify-content: space-between; align-items: center; margin-bottom: 8px;
}
.modal-header h2 { font-size: 20px; }
.icon-btn { background: none; color: #6b7280; font-size: 24px; padding: 0 8px; line-height: 1; }
.modal-description { color: #4b5563; margin-bottom: 12px; }
.modal-body label { display: block; font-weight: 600; margin: 8px 0 4px; }
.modal-body input { width: 100%; padding: 8px 10px; border: 1px solid #d1d5db; border-radius: 6px; font-
size: 15px; }
.modal-body input[aria-invalid="true"] { border-color: #dc2626; }
.error { color: #dc2626; font-size: 14px; margin-top: 4px; }
.modal-actions { display: flex; justify-content: flex-end; gap: 8px; margin-top: 16px; }

@keyframes fade-in { from { opacity: 0; } }
@keyframes pop-in { from { opacity: 0; transform: translateY(8px) scale(0.98); } }
@media (prefers-reduced-motion: reduce) { .backdrop, .modal { animation: none; } }
```

src/index.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

`src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled: reliable backdrop detection (no class-name checks, no close on drag-out), Escape, focus in / trapped / restored, portal stacking, scroll lock without layout shift, inert background, stale-callback-free effects, no re-focus while typing, destructive confirms (no backdrop close, focus on Cancel), long content, reduced motion, StrictMode.

Trade-offs to mention to the interviewer

- *Custom modal vs native* `<dialog>` + `showModal()` *:* the native element gives the top layer, Escape, an inert background and basic focus handling for free, and is the best production choice for most cases. A custom one gives full control and shows you understand what's needed (the interview point).
- *Manual focus trap vs* `inert` *alone:* `inert` already blocks the page; the Tab trap adds wrap-around inside the modal instead of jumping to the browser chrome.
- *Controlled (* `isOpen` *prop) vs internal state:* controlled lets the parent decide, e.g. block closing with unsaved changes. Internal state is simpler for fire-and-forget dialogs.
- *Unmount on close vs keep mounted:* unmounting resets form state and saves memory; keeping it mounted allows close animations and keeps the draft.
- *No close animation:* exit animations need delayed unmounting (an `isClosing` state, or `animationend`); worth it in production.

What to add with more time

- Exit animation with delayed unmount; `@starting-style` / View Transitions for `<dialog>`.
- Nested modals: a stack so Escape closes only the top one; only the top modal traps focus.
- "Unsaved changes?" guard: `onClose` checks a dirty flag.
- A `useModal()` hook / context for opening modals imperatively (`confirm({ title })` returning a promise).

- Tests with React Testing Library + `user-event`: Tab wrap, Escape, focus restore, backdrop rules, `aria-*` attributes.

13. Counter questions the interviewer may ask

Requirements and design

Q1. Why render the modal in a portal?

So it's a child of `<body>`, not of whatever component opened it. Parents with `overflow: hidden`, `transform`, `filter` or their own `z-index` stacking context can clip or bury a `position: fixed` element. React events still bubble through the React tree (not the DOM tree), so handlers keep working.

Q2. Why not just use the native `<dialog>` element?

In production I often would: `dialog.showModal()` puts it in the browser's top layer, makes the rest of the page inert, closes on Escape (`cancel` event) and handles basic focus. You still need: focus restore checks, backdrop-click handling (`::backdrop` clicks hit the dialog element), scroll lock, and React integration (`useEffect` calling `showModal` / `close`).

Q3. Controlled or uncontrolled modal?

Controlled (`isOpen` + `onClose`): the parent decides when it opens and can refuse to close (unsaved changes, a request in progress). Uncontrolled (a trigger prop and internal state) is simpler but harder to coordinate with app logic.

State design

Q4. Why one `openModal` value instead of a boolean per modal?

It makes "two modals open at once" impossible, and adding a modal adds a value, not another `useState`. For real stacking (a confirm on top of a form) use an array/stack instead.

Q5. Where does the form draft live and why?

Inside `EditProfileForm`, which only mounts while the modal is open. Opening the modal again starts from the saved name; Cancel needs no reset code. If the draft must survive closing (accidental close), lift it up or keep the modal mounted.

Q6. Why the `onCloseRef` pattern?

`onClose` is usually a new inline function each render. With it in the effect's dependency list, the effect re-runs on every render: focus jumps back to the first field while the user is typing. The ref always holds the latest callback, while the effect depends only on `isOpen`.

Hard parts

Q7. How does the focus trap work? What are its limits?

On Tab at the last focusable element → focus the first; Shift+Tab at the first → focus the last. Limits: the selector may miss focusable things (`contenteditable`, `audio controls`, elements hidden with CSS), and

dynamic content changes the list, so query it at keydown time (as we do). Libraries like `focus-trap` handle more cases; `inert` on the background covers most of the rest.

Q8. What does `inert` do, and what did people use before it?

`inert` makes an element and its subtree non-clickable, non-focusable and hidden from the accessibility tree. Before it, people set `aria-hidden="true"` on siblings + a JS focus trap + `pointer-events: none`. It's supported in all modern browsers.

Q9. How do you lock background scrolling, and what's the catch?

`body { overflow: hidden }` while open, restoring the previous value on close. Catches: the scrollbar disappears and the page shifts sideways (fix: pad by the scrollbar width, or `scrollbar-gutter: stable`), and iOS Safari may still scroll (fix: `position: fixed` on body with a saved scroll position, or `overscroll-behavior: contain` on the modal).

Q10. Why check `e.target === e.currentTarget` instead of a class name?

`currentTarget` is the element with the handler (the backdrop), and `target` is the element actually clicked. If they're equal, the click was on the backdrop itself, not bubbling from inside the modal. Class-name checks break when classes change or multiple classes are present.

Q11. How would you support nested modals?

Keep a modal stack (context or module-level array). Only the top modal handles Escape and traps focus; each closing modal restores focus to the element that opened it (which is inside the modal below). With `inert`, mark the lower modals inert too.

Implementation steps

Q12. Where should focus go when a modal opens?

To the most useful element: the first input of a form, the **least destructive** button in a confirm (Cancel), or the dialog/title itself for long content (so screen readers start at the top). Never leave it on the page behind.

Q13. Which ARIA attributes does a dialog need?

`role="dialog"` (or `alertdialog` for urgent confirms), `aria-modal="true"`, `aria-labelledby` pointing to the title, optionally `aria-describedby` for the description. The close button needs an accessible name (`aria-label="Close"`). `useId` generates unique ids so several modals don't clash.

Q14. How would you test this modal?

React Testing Library + `user-event`: open → expect the dialog by role and name, focus on the input; `user.tab()` from the last element → focus on the first; `user.keyboard("{Escape}")` → dialog gone and focus on the trigger; click the backdrop → closed (or not, for the confirm); check `inert` / body overflow are cleaned up.

Q15. React StrictMode runs effects twice in development. Does your effect survive that?

Yes, because the cleanup fully undoes everything (listener removed, styles restored, inert removed, focus restored). The second run then starts from a clean state. Effects that aren't fully reversible show bugs in StrictMode, which is exactly why it double-runs them.

Performance and scale

Q16. The app has 50 different modals. How do you organise them?

A modal manager: one `ModalProvider` with a stack in context and a `useModal()` hook (`open(<Content/>, options)`), or an imperative `confirm()` that returns a promise. The modals render in one portal root, and code-split heavy modal contents with `React.lazy` .

Q17. Should closed modals stay mounted?

Usually no: unmounting saves memory, resets state and removes listeners. Keep them mounted (hidden) only when you need instant reopen, preserved state, or exit animations, and then make sure they're `inert` / `hidden` while closed.

Q18. How do you add a close animation?

Don't unmount immediately: set `isClosing` , play the CSS animation, and unmount on `animationend` (or after the duration). Also respect `prefers-reduced-motion` . With `<dialog>` , CSS `@starting-style` and `transition-behavior: allow-discrete` can animate both directions.

Trade-offs and "why not X?"

Q19. Why not a library (Radix Dialog, Headless UI, MUI Modal)?

In production, yes: they handle focus traps, scroll lock (including iOS), nested dialogs, portals, SSR and ARIA, and are well tested. In an interview, building it shows you know *what* those libraries solve.

Q20. Why not `alert()` / `confirm()` for the delete confirmation?

They block the main thread, can't be styled, can't show rich content, are suppressed in some contexts (iframes, repeated prompts), and look different in every browser. A custom confirm is consistent and accessible, and can be awaited with a promise-based API.

Q21. What's the difference between a modal and a non-modal dialog (popover)?

A modal blocks the rest of the page until it's dismissed (focus trapped, background inert). A non-modal dialog / popover lets users keep interacting with the page (no trap, closes on outside click or Escape). Use `aria-modal` only for real modals; the HTML `popover` attribute covers many non-modal cases.